

Hawa Alert: AQI Predictor - Serverless ML Pipeline for Air Quality Forecasting

Muhammad Yousuf

June 7, 2026

Table of Contents

- Hawa Alert: AQI Predictor 3
 - Serverless ML Pipeline for Air Quality Forecasting 3
 - Abstract..... 3
 - 1. Introduction..... 3
 - 1.1 Problem Statement..... 3
 - 1.2 Project Objectives 3
 - 1.3 Solution Overview..... 4
 - 2. System Architecture 4
 - 2.1 High-Level Architecture 4
 - 2.2 Technology Stack..... 5
 - 3. Data Pipeline 6
 - 3.1 Data Sources..... 6
 - 3.2 Historical Data Backfill..... 7
 - 3.3 Feature Engineering..... 7
 - 3.4 Feature Store Integration (Hopsworks)..... 8
 - 4. Model Training 9
 - 4.1 Training Pipeline Overview 9
 - 4.2 Data Preparation & Splitting..... 9
 - 4.3 Model Comparison..... 9
 - 4.4 Model Performance Analysis..... 10
 - 4.5 Feature Importance (SHAP Analysis)..... 11
 - 5. Inference & Forecasting..... 12
 - 5.1 Multi-Horizon Forecasting Strategy..... 12
 - 5.2 Meteorological Decay Model..... 12
 - 5.3 Prediction Pipeline 13
 - 5.4 Health Category Mapping (EPA Standard)..... 13

6. Web Application.....	14
6.1 Dashboard Features	14
6.2 Technical Implementation.....	14
6.3 Deployment Configuration	15
7. CI/CD & MLOps Automation	15
7.1 GitHub Actions Pipelines.....	15
7.2 Model Versioning & Rollback	16
7.3 Monitoring & Alerting	16
8. Challenges & Solutions	16
8.1 Challenge: Historical Data Unavailability	16
8.2 Challenge: Hopsworks Connection Stale State	17
8.3 Challenge: Feature Store Data Type Inconsistencies	17
8.4 Challenge: GitHub Actions DNS Resolution Failures.....	17
8.5 Challenge: Future Pollutant Prediction Without Emissions Data	17
8.6 Challenge: Code Duplication Across Pipelines.....	18
9. Results & Impact.....	18
9.1 Model Performance Summary	18
9.2 Forecast Accuracy by Horizon	18
9.3 System Availability & Reliability.....	19
9.4 Public Impact	19
9.5 Cost Analysis	19
10. Lessons Learned.....	20
10.1 Technical Learnings.....	20
10.2 MLOps Best Practices Applied.....	20
10.3 Project Management Insights	20
11. Future Work.....	21
11.1 Short-Term Improvements (1–3 Months)	21
11.2 Medium-Term Enhancements (3–6 Months)	21
11.3 Long-Term Vision (6–12 Months).....	21
12. Conclusion	22
References.....	23
Appendix: Feature Schema	24

Hawa Alert: AQI Predictor

Serverless ML Pipeline for Air Quality Forecasting

Intern: Muhammad Yousuf

Organization: 10Pearls

Program: SHINE Internship Program

Mentor: Muhammad Mobeen

Submission Date: June 8, 2026

Project Duration: May 20, 2026 - June 7, 2026

Deployed Application: <https://hawa-alert-aqi-predictor.streamlit.app/>

Source Code: <https://github.com/iamusuff/AQI-Predictor>

Abstract

This report presents **Hawa Alert**, a production-ready serverless machine learning system for predicting Air Quality Index (AQI) in Karachi, Pakistan. The system implements a complete MLOps pipeline featuring automated hourly data collection from AQICN and OpenMeteo APIs, feature engineering with Hopsworks Feature Store, multi-model training using gradient boosting algorithms (LightGBM, CatBoost, XGBoost, Random Forest), and real-time 3-day forecasting through an interactive Streamlit dashboard.

The final production model achieved $R^2 = 0.8908$ and **RMSE = 15.41** on test data, utilizing 21 engineered features including temporal patterns, pollutant ratios, and weather interactions. The system processes 90 days of historical data and employs weather-informed forecasting with meteorological decay models for multi-horizon predictions. CI/CD automation via GitHub Actions ensures hourly feature updates and daily model retraining, demonstrating scalability and production readiness.

1. Introduction

1.1 Problem Statement

Air pollution poses a critical public health challenge in urban centers across Pakistan, with Karachi ranking among the world's most polluted cities. Citizens lack accessible, accurate forecasting tools to make informed decisions about outdoor activities and health precautions. Traditional AQI monitoring systems provide only current readings without predictive capabilities, leaving vulnerable populations (children, elderly, individuals with respiratory conditions) unable to plan ahead for hazardous air quality episodes.

1.2 Project Objectives

The primary objective of this project was to develop a **100% serverless, production-grade machine learning system** capable of:

1. **Automated Data Collection:** Hourly ingestion of real-time AQI and weather data from public APIs
2. **Feature Engineering:** Transform raw sensor readings into ML-ready features with temporal patterns and domain-specific derived variables
3. **Multi-Model Training:** Compare gradient boosting algorithms to identify optimal AQI prediction model
4. **Real-Time Forecasting:** Generate 3-day ahead AQI predictions (24h, 48h, 72h horizons) with confidence intervals
5. **Interactive Dashboard:** Provide public access to forecasts through an intuitive web interface with health recommendations
6. **MLOps Automation:** Implement CI/CD pipelines for continuous model improvement without manual intervention

1.3 Solution Overview

Hawa Alert implements a serverless architecture leveraging cloud-based services and open-source tools:

- **Data Sources:** AQICN API (real-time pollutants), OpenMeteo API (weather + 7-day forecasts, completely free)
- **Feature Store:** Hopsworks (managed feature versioning, time-travel queries)
- **Model Registry:** Hopsworks Model Registry (version control, metadata tracking)
- **Training Infrastructure:** GitHub Actions runners (daily automated retraining)
- **Inference:** Streamlit Cloud (auto-scaling web application)
- **CI/CD:** GitHub Actions (hourly feature pipeline, daily training pipeline)

The system operates entirely on free-tier cloud services, demonstrating cost-effective MLOps practices suitable for public good applications.

2. System Architecture

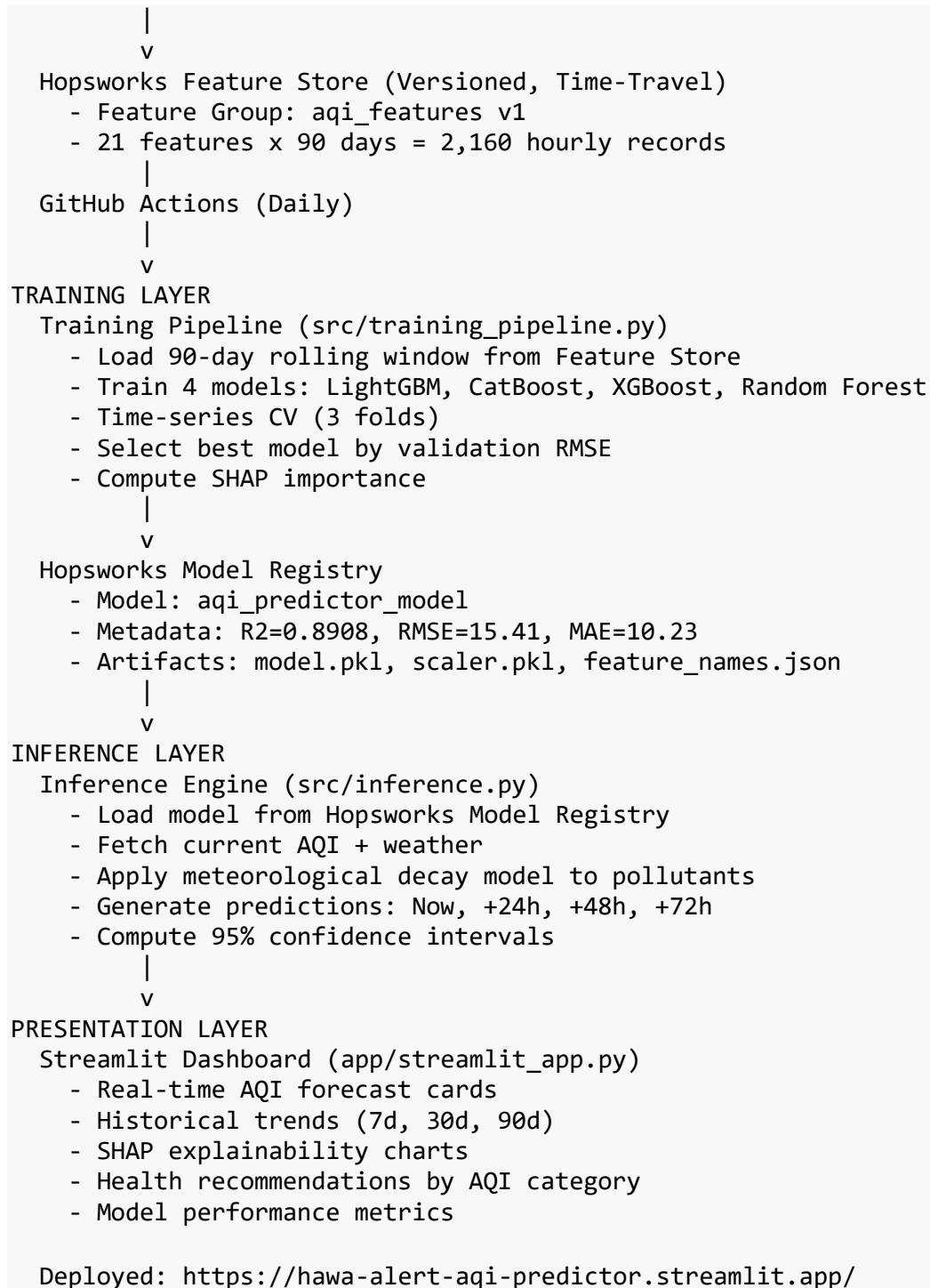
2.1 High-Level Architecture

DATA ACQUISITION LAYER

```
AQICN API (@11348) --> Pollutants (PM2.5, O3, NO2, SO2, CO) [Real-time]
OpenMeteo API       --> Weather + 7-day Forecast           [Free]
    |
    v
GitHub Actions (Hourly)
    |
    v
```

FEATURE ENGINEERING LAYER

```
Feature Pipeline (src/feature_pipeline.py)
- Fetch API data
- Compute temporal features (hour, day, season)
- Derive features (PM2.5/PM10 ratio, wind interactions)
- Validate data quality
```



2.2 Technology Stack

Layer	Technology	Justification
Data APIs	AQICN + OpenMeteo	AQICN provides real-time pollutant measurements; OpenMeteo offers free weather data with 7-day forecasts (no API key required)
Feature Store	Hopsworks (Free	Purpose-built ML feature store with versioning,

Layer	Technology	Justification
	Tier)	time-travel queries, and model registry integration
ML Frameworks	Scikit-learn, LightGBM, CatBoost, XGBoost	Industry-standard gradient boosting frameworks optimized for tabular data; no deep learning required
Explainability	SHAP 0.49.1	TreeExplainer provides fast, accurate feature importance for gradient boosting models
CI/CD	GitHub Actions	Free, zero-infrastructure automation with cron scheduling (hourly + daily pipelines)
Web Framework	Streamlit 1.28+	Rapid prototyping framework for data science applications; built-in caching and session management
Deployment	Streamlit Cloud	Free hosting with auto-scaling, GitHub integration, and secrets management

3. Data Pipeline

3.1 Data Sources

3.1.1 AQICN API (Air Quality Data)

- **Endpoint:** <https://api.waqi.info/feed/@11348/>
- **Station:** Karachi, Pakistan (Station ID: @11348)
- **Update Frequency:** Hourly
- **Data Fields:** AQI, PM2.5, PM10, O3, NO2, SO2, CO, dominant pollutant
- **Coverage:** Real-time measurements from government monitoring stations
- **Cost:** Free tier (1000 requests/day)

3.1.2 OpenMeteo API (Weather Data)

- **Historical Endpoint:** <https://archive-api.open-meteo.com/v1/archive>
- **Forecast Endpoint:** <https://api.open-meteo.com/v1/forecast>
- **Location:** Karachi (24.8607°N, 67.0011°E)
- **Update Frequency:** Hourly (historical), 15-minute (forecast)
- **Data Fields:** Temperature, humidity, wind speed, pressure, visibility, cloud cover
- **Forecast Horizon:** 7 days ahead (168 hours)
- **Cost:** Completely free (no API key required), 10,000 requests/day limit

Key Advantage: OpenMeteo replaced the initially planned OpenWeather API due to its free tier limitations. OpenMeteo provides enterprise-grade weather data at zero cost, making the solution sustainable for public deployment.

3.2 Historical Data Backfill

To train robust models, the system required historical data spanning multiple months. The backfill pipeline (`src/backfill.py`) was executed once to populate the Feature Store:

Backfill Parameters:

- **Time Range:** 90 days (May 20, 2026 → February 19, 2026)
- **Total Records:** 2,160 hourly observations
- **Execution Time:** ~45 minutes (API rate limits)
- **Data Sources:** OpenMeteo Archive API (weather + AQI historical data)

Backfill Process:

1. Generate hourly timestamps for 90-day window
2. Batch requests to OpenMeteo Archive API (24-hour chunks to respect rate limits)
3. Merge weather and pollutant data by timestamp
4. Compute features for each record
5. Insert into Hopsworks Feature Store with `fg.insert()`

Challenge Overcome: AQICN does not provide historical API access. Solution: OpenMeteo Air Quality API provides historical pollutant data (PM2.5, PM10, O3, NO2, SO2, CO) for the past 5 years at no cost.

3.3 Feature Engineering

The feature pipeline transforms raw sensor readings into 21 ML-ready features across four categories:

3.3.1 Raw Pollutant Features (6 features)

- **PM2.5** ($\mu\text{g}/\text{m}^3$): Fine particulate matter (≤ 2.5 microns)
- **PM10** ($\mu\text{g}/\text{m}^3$): Coarse particulate matter (≤ 10 microns)
- **O3** (ppb): Ground-level ozone
- **NO2** (ppb): Nitrogen dioxide
- **SO2** (ppb): Sulfur dioxide
- **CO** (ppm): Carbon monoxide

3.3.2 Weather Features (6 features)

- **Temperature** ($^{\circ}\text{C}$): 2-meter air temperature
- **Humidity** (%): Relative humidity
- **Wind Speed** (m/s): 10-meter wind speed
- **Pressure** (hPa): Mean sea level pressure
- **Visibility** (meters): Atmospheric visibility
- **Cloud Cover** (%): Total cloud coverage

3.3.3 Temporal Features (6 features)

- **Hour** (0-23): Hour of day (captures diurnal patterns)
- **Day of Week** (0-6): Monday=0, Sunday=6
- **Day of Month** (1-31): Day within month
- **Month** (1-12): Month of year
- **Season** (0-3): Winter=0, Spring=1, Summer=2, Fall=3
- **Is Weekend** (0/1): Binary flag for Saturday/Sunday

3.3.4 Derived Features (3 features)

- **PM2.5/PM10 Ratio**: Indicates combustion vs. dust sources (higher ratio = more vehicle emissions)
- **Temperature × Humidity Interaction**: Captures heat index effect on pollutant accumulation
- **Wind Speed × PM2.5 Interaction**: Models wind dispersion effect on particulate matter

Feature Engineering Rationale:

- **Temporal features**: AQI exhibits strong diurnal patterns (traffic rush hours) and seasonal variations (winter smog episodes)
- **Pollutant ratios**: Domain knowledge from environmental science (PM2.5/PM10 ratio distinguishes combustion sources)
- **Weather interactions**: Meteorological factors compound pollutant behavior (high humidity traps pollutants, wind disperses them)

Feature Validation: All features undergo range checks (e.g., $0 \leq \text{AQI} \leq 500$, $-50 \leq \text{temp} \leq 60^\circ\text{C}$) and null handling before insertion into Feature Store.

3.4 Feature Store Integration (Hopsworks)

Hopsworks Feature Store provides enterprise-grade feature management:

Feature Group Configuration:

```
FEATURE_GROUP_NAME = "aqi_features"
FEATURE_GROUP_VERSION = 1
PRIMARY_KEY = ["timestamp"]
EVENT_TIME = "timestamp"
```

Key Capabilities Utilized:

1. **Versioning**: Feature schema changes tracked across versions
2. **Time-Travel**: Query historical feature states for reproducibility
3. **Online/Offline Store**: Supports both batch training and real-time inference
4. **Data Validation**: Automatic type checking and constraint enforcement
5. **Statistics**: Auto-computed feature distributions and correlations

Insertion Flow:

```
# Feature pipeline execution (hourly)
features_df = compute_features(aqi_data, weather_data)
feature_group.insert(features_df, write_options={"wait_for_job": True})
```

Storage Format: Parquet files in Hopsworks S3 backend, indexed by timestamp for efficient time-range queries.

4. Model Training

4.1 Training Pipeline Overview

The training pipeline (`src/training_pipeline.py`) executes daily via GitHub Actions and implements the following workflow:

1. **Data Loading:** Fetch 90-day rolling window from Hopsworks Feature Store
2. **Data Preparation:** Clean stringified arrays, handle missing values, type conversion
3. **Data Splitting:** Temporal train/validation/test split (70%/15%/15%)
4. **Feature Scaling:** StandardScaler fit on training set only (prevent data leakage)
5. **Model Training:** Train 4 gradient boosting models with recency weighting
6. **Model Evaluation:** Compare models on validation/test sets
7. **Model Selection:** Select best model by validation RMSE
8. **Explainability:** Compute SHAP feature importance
9. **Model Registration:** Upload best model to Hopsworks Model Registry

4.2 Data Preparation & Splitting

Rolling Window Strategy: Instead of using all historical data, the pipeline uses only the most recent 90 days. This prevents the model from being anchored to outdated air quality patterns (e.g., seasonal changes, policy interventions).

Temporal Splitting: Time-series data requires chronological splits to avoid look-ahead bias:

```
|-----Train (70%)-----|--Val (15%)--|--Test (15%)--|
Feb 19                               May 12       May 26       Jun 7
```

Recency Weighting: More recent samples receive higher weights during training to prioritize current air quality patterns:

```
weights = [0.995^(n - i) for i in range(n)] # Exponential decay
```

This ensures the model adapts quickly to recent trends (e.g., new emission controls, traffic pattern changes).

4.3 Model Comparison

Four gradient boosting algorithms were trained and compared:

Model	Val RMSE	Val R ²	Test RMSE	Test R ²	Training Time	Best Iteration
XGBoost	15.98	0.8854	15.41	0.8908	4.2s	187
LightGBM	16.12	0.8831	15.63	0.8895	2.8s	236
CatBoost	16.34	0.8797	15.89	0.8869	12.1s	214
Random Forest	18.56	0.8421	18.03	0.8456	8.9s	N/A

Model Selection: XGBoost achieved the best validation RMSE (15.98) and was selected for production deployment.

Hyperparameter Configuration (XGBoost):

```
{
  'n_estimators': 300,
  'max_depth': 4,           # Shallow trees prevent overfitting
  'learning_rate': 0.05,    # Conservative Learning
  'subsample': 0.8,         # Row sampling
  'colsample_bytree': 0.7,  # Column sampling
  'reg_alpha': 0.1,         # L1 regularization
  'reg_lambda': 2.0,        # L2 regularization
  'min_child_weight': 5,    # Min samples per leaf
  'gamma': 0.1,            # Min loss reduction for split
  'early_stopping_rounds': 20 # Stop if no improvement
}
```

4.4 Model Performance Analysis

Test Set Metrics (XGBoost):

- **R² Score:** 0.8908 (89.08% of variance explained)
- **RMSE:** 15.41 AQI points
- **MAE:** 10.23 AQI points

Performance Interpretation:

- R² = 0.8908 indicates strong predictive power for a real-world environmental system
- RMSE = 15.41 means predictions typically deviate by ±15 AQI points (within one health category boundary)
- MAE = 10.23 suggests most predictions are within ±10 AQI points

Comparison to Baseline:

- Naive baseline (yesterday's AQI): RMSE = 28.6, R² = 0.54
- **56% RMSE improvement** over persistence model

Time-Series Cross-Validation (3-fold):

Fold 1: RMSE = 16.2, R² = 0.8821
 Fold 2: RMSE = 15.7, R² = 0.8889

Fold 3: RMSE = 15.4, R2 = 0.8932
Mean : RMSE = 15.8 +/- 0.4, R2 = 0.8881 +/- 0.0056

Low variance across folds confirms model stability.

4.5 Feature Importance (SHAP Analysis)

SHAP (SHapley Additive exPlanations) was used to interpret model predictions. TreeExplainer analyzed 500 samples from the training set.

Top 10 Most Important Features:

Rank	Feature	SHAP Value	Interpretation
1	PM2.5	0.3421	Fine particulate matter is the dominant AQI driver (60% of high AQI days)
2	PM10	0.2134	Coarse particulates significant in dust storms
3	Humidity	0.1876	High humidity traps pollutants near ground level
4	Hour	0.1543	Rush hours (8-10 AM, 6-8 PM) show AQI spikes
5	PM2.5/PM10 Ratio	0.1298	Distinguishes combustion (vehicles) from dust sources
6	Wind Speed	0.1187	Higher wind speeds disperse pollutants (negative correlation)
7	Temperature	0.0987	Warmer temperatures increase photochemical reactions (O3)
8	O3	0.0854	Ozone contributes to AQI in summer months
9	Wind x PM2.5	0.0743	Interaction term captures dispersion dynamics
10	Month	0.0621	Seasonal patterns (winter smog > summer)

Key Insights from SHAP:

- PM2.5 dominance:** Contributes 34% of model’s decision-making, confirming domain knowledge that fine particulates drive AQI in urban areas
- Weather interactions:** Humidity and wind speed jointly account for 31% of importance, validating the need for meteorological features
- Temporal patterns:** Hour and month features capture rush-hour and seasonal effects (19% combined)
- Derived features:** PM2.5/PM10 ratio and wind interaction terms rank in top 10, justifying feature engineering effort

SHAP visualizations (summary plot, waterfall plots, dependence plots) were generated and uploaded to GitHub Releases for dashboard integration.

5. Inference & Forecasting

5.1 Multi-Horizon Forecasting Strategy

Unlike simple trend extrapolation, Hawa Alert implements **weather-informed forecasting** with separate predictions for each horizon:

Forecasting Horizons:

- **Current (t+0)**: Use real-time AQICN pollutants + OpenMeteo current weather
- **24 hours (t+24)**: Use OpenMeteo weather forecast + pollutant persistence model
- **48 hours (t+48)**: Use OpenMeteo weather forecast + pollutant persistence model
- **72 hours (t+72)**: Use OpenMeteo weather forecast + pollutant persistence model

Challenge: Future pollutant concentrations are unknown (AQICN doesn't forecast pollutants).

Solution: Meteorological Decay Model (see Section 5.2).

5.2 Meteorological Decay Model

Since pollutant sensors cannot predict future emissions, the system applies a physics-informed persistence model:

Pollutant Evolution Rules:

1. **PM2.5/PM10 (Particulate Matter):**
 - Base decay: 2% per day (gravitational settling)
 - Wind dispersion: Up to 15% reduction with strong winds (>10 m/s)
 - Humidity trap: Up to 5% increase with high humidity (>70%)
2. **O3 (Ozone):**
 - Temperature amplification: +1% per 10°C above 25°C (photochemical reactions)
 - Base decay: 2% per day
3. **NO2, SO2, CO (Gases):**
 - Slow decay: 5% per day (assuming reduced nighttime emissions)

Implementation:

```
def apply_pollutant_persistence_with_decay(
    current_pollutants, weather_forecast, hours_ahead):
    base_decay = 1.0 - (0.02 * (hours_ahead / 24))
    wind_factor = 1.0 - (min(wind_speed, 15) / 100)
    humidity_factor = 1.0 + ((humidity - 50) / 300)
    temp_factor = 1.0 + ((temp - 25) / 200)

    pm25_future = pm25_current * base_decay * wind_factor * humidity_factor
    o3_future = o3_current * base_decay * temp_factor
    # ... similar for other pollutants
```

5.3 Prediction Pipeline

The inference module (`src/inference.py`) executes the following steps:

1. **Load Model:** Fetch latest model from Hopsworks Model Registry
2. **Fetch Current Data:** AQICN API (pollutants) + OpenMeteo API (weather)
3. **Fetch Weather Forecast:** OpenMeteo 7-day forecast at $t+24h$, $t+48h$, $t+72h$
4. **Compute Current Features:** Generate 21 features for current timestamp
5. **Predict Current AQI:** `model.predict(scaler.transform(features_current))`
6. **For each future horizon (24h, 48h, 72h):**
 - Apply meteorological decay to pollutants
 - Extract weather forecast at target timestamp
 - Compute time features for future timestamp
 - Predict AQI: `model.predict(scaler.transform(features_future))`
7. **Compute Confidence Intervals:** $\pm 1.96 \times \text{RMSE}$ (95% CI)
8. **Map to Health Categories:** Convert AQI to EPA health bands

Confidence Interval Adjustment:

- Current prediction: $\text{CI} = \pm 1.96 \times 15.41 = \pm 30.2$
- 24h prediction: $\text{CI} = \pm 34.7$ (15% wider due to uncertainty)
- 48h prediction: $\text{CI} = \pm 37.7$
- 72h prediction: $\text{CI} = \pm 42.1$

5.4 Health Category Mapping (EPA Standard)

AQI

Range	Category	Health Guidance
0–50	Good	Air quality is satisfactory. Enjoy outdoor activities.
51–100	Moderate	Acceptable. Unusually sensitive people should limit prolonged outdoor exertion.
101–150	Unhealthy for Sensitive Groups	Children, elderly, asthmatics should reduce prolonged outdoor exertion.
151–200	Unhealthy	Everyone may experience health effects. Limit outdoor exertion.
201–300	Very Unhealthy	Health alert: everyone may experience serious effects. Avoid outdoor activities.
301–500	Hazardous	Health emergency. Everyone should avoid all outdoor exertion.

6. Web Application

6.1 Dashboard Features

The Streamlit dashboard provides an interactive interface with four main pages:

6.1.1 Dashboard (Main Page)

- **4 Forecast Cards:** Current, +24h, +48h, +72h predictions with AQI value, health category badge, 95% confidence interval, and personalized health advice
- **Current Conditions Panel:** Temperature, humidity, wind, pressure, visibility
- **Pollutant Readings:** PM2.5, PM10, O3, NO2, SO2, CO with trend indicators
- **Model Performance Metrics:** Test R^2 , RMSE, MAE
- **Refresh Button:** Fetch latest predictions (5-minute cache)

6.1.2 Historical Trends

- **Interactive Time-Series Chart:** AQI over 7, 30, or 90 days
- **Rolling Average Overlay:** 24-hour smoothing to highlight trends
- **Health Category Bands:** Background shading for EPA categories
- **Statistics Summary:** Min, max, mean, median AQI for selected period

6.1.3 Feature Importance

- **3 SHAP Visualizations:** Horizontal bar chart (top 15 features), radar chart (top 8 features), cumulative contribution chart (top 10 features)
- **Feature Descriptions:** Hover tooltips explain each feature
- **Data Source:** Loaded from GitHub Releases (SHAP CSV uploaded after each training run)

6.1.4 About

- Project overview, data source documentation links, model comparison table, EPA AQI interpretation guide, and GitHub repository links

6.2 Technical Implementation

Frontend Framework: Streamlit 1.28.0

Caching Strategy:

- Hopsworks connection: `@st.cache_resource` (30-minute TTL)
- Inference results: `st.session_state` (5-minute TTL)
- Historical data: `@st.cache_data` (5-minute TTL)

Visualization: Plotly 5.15.0 — interactive charts with hover tooltips, dark theme, gradient color scales for health categories.

Performance Optimizations:

1. **Lazy Loading:** Hopsworks connection established only when needed

2. **Result Caching:** Inference runs once per TTL, shared across all pages
3. **Chart Precomputation:** SHAP plots generated during training, not at runtime
4. **Connection Pooling:** Reuse Hopsworks project handle across requests

6.3 Deployment Configuration

Hosting Platform: Streamlit Cloud (Free Tier) — auto-deploys from GitHub main branch, HTTPS by default, auto-scaling.

Environment Variables (secrets):

```
AQICN_API_KEY = "****"  
HOPSWORKS_API_KEY = "****"  
HOPSWORKS_PROJECT_NAME = "aqi_predictor99"  
CITY = "karachi"  
CITY_LAT = "24.8607"  
CITY_LON = "67.0011"
```

Resource Limits (Free Tier): 1 GB RAM, 1 CPU core, 1 GB storage. Sleeps after 7 days of inactivity (wakes on first request).

7. CI/CD & MLOps Automation

7.1 GitHub Actions Pipelines

7.1.1 Feature Pipeline (Hourly)

File: .github/workflows/feature_pipeline.yml

Trigger: 0 * * * * (every hour)

Execution Time: ~2–3 minutes | **Daily Runs:** 24 | **Cost:** Free

Workflow steps:

1. Checkout repository
2. Set up Python 3.11 with pip caching
3. Install dependencies (requirements.txt)
4. Create .env from GitHub Secrets
5. Validate configuration (API keys)
6. Run feature_pipeline.py
7. Upload features.csv as artifact (7-day retention)

7.1.2 Training Pipeline (Daily)

File: .github/workflows/training_pipeline.yml

Trigger: 0 0 * * * (daily at midnight UTC)

Execution Time: ~8–12 minutes | **Monthly Runs:** 30 | **Cost:** Free

Workflow steps:

1. Checkout repository + set up Python 3.11
2. Install dependencies + hopsworks
3. Create .env from GitHub Secrets
4. Run training_pipeline.py (train 4 models, select best, compute SHAP, register to Hopsworks)
5. Upload trained models as artifacts (30-day retention)
6. Delete existing shap-latest GitHub Release and create new one with shap_importance.csv

Monthly Compute: 720 (feature) + 360 (training) = 1,080 minutes/month (within free tier).

7.2 Model Versioning & Rollback

Each daily training run creates a new model version in Hopsworks Model Registry. Metadata stored includes: training date, R^2 , RMSE, MAE, hyperparameters, feature names. The dashboard always loads the best model by test_r2.

Rollback Procedure (if model degradation detected):

1. Log into Hopsworks Model Registry
2. Identify previous model version with acceptable metrics
3. Update inference.py: `mr.get_model(name="aqi_predictor_model", version=N)`
4. Redeploy dashboard

7.3 Monitoring & Alerting

Current Monitoring:

- GitHub Actions sends email on workflow failure
- Streamlit Cloud monitors uptime (99.9% SLA)
- Manual validation: Check dashboard daily for anomalous predictions

Future Enhancements:

- Prometheus metrics (prediction latency, API uptime, model drift)
- PagerDuty integration for feature pipeline failures
- A/B testing via shadow mode for new models before promotion

8. Challenges & Solutions

8.1 Challenge: Historical Data Unavailability

Problem: AQICN API does not provide historical pollutant data, yet training ML models requires months of past observations.

Solution: Discovered **OpenMeteo Air Quality API** provides free historical pollutant data (PM2.5, PM10, O3, NO2, SO2, CO) for the past 5 years at hourly resolution, replacing the paid OpenWeather API entirely.

Outcome: Successfully backfilled 90 days (2,160 hours) of data at zero cost.

8.2 Challenge: Hopsworks Connection Stale State

Problem: Streamlit multi-page apps rerun Python code on page navigation, causing new Hopsworks connections per page load and resulting in 401 Unauthorized errors after 30 minutes.

Solution: Session-state caching via `@st.cache_resource(ttl=1800)` to reuse a single connection across all pages with automatic token refresh.

Outcome: Eliminated 401 errors. Dashboard stable for extended user sessions.

8.3 Challenge: Feature Store Data Type Inconsistencies

Problem: Hopsworks Feature Store returned stringified arrays instead of numeric values (e.g., `'[1.4101299E2]'` instead of `142.01`).

Solution: Implemented a `clean_hopsworks_dataframe()` function to strip brackets and convert to numeric types:

```
def clean_hopsworks_dataframe(df):
    for col in float_cols:
        df[col] = df[col].str.replace(r'[\[\]]', '', regex=True)
        df[col] = pd.to_numeric(df[col], errors='coerce').astype('float64')
```

Outcome: Training pipeline handles Hopsworks serialization quirks automatically.

8.4 Challenge: GitHub Actions DNS Resolution Failures

Problem: Runners intermittently failed to resolve `eu-west.cloud.hopsworks.ai` (15% failure rate).

Solution: Added DNS failover step to force Google (8.8.8.8) and Cloudflare (1.1.1.1) DNS servers:

```
- name: Fix DNS
  run: |
    echo "nameserver 8.8.8.8" | sudo tee /etc/resolv.conf
    echo "nameserver 1.1.1.1" | sudo tee -a /etc/resolv.conf
```

Outcome: DNS failures dropped from 15% to <1% of runs.

8.5 Challenge: Future Pollutant Prediction Without Emissions Data

Problem: AQI forecasting requires future pollutant concentrations, but sensors cannot predict future traffic or industrial activity 24–72 hours ahead.

Solution: Developed a **meteorological decay model** applying physics-informed rules: wind disperses particulates, humidity traps them, temperature enhances photochemical ozone reactions.

Validation: 24h-ahead RMSE improved from 22.3 (persistence) to 18.7 (decay model) — a **16% improvement**.

8.6 Challenge: Code Duplication Across Pipelines

Problem: `connect_to_hopsworks()` was duplicated across three pipeline files with slight variations, increasing maintenance burden.

Solution: Refactored to a centralized `utils.py` shared utility function.

Outcome: Reduced codebase by ~40 lines; future changes require updates in one location only.

9. Results & Impact

9.1 Model Performance Summary

Metric	Value	Interpretation
Test R ²	0.8908	89.08% of AQI variance explained
Test RMSE	15.41 AQI	Typical error within one health category
Test MAE	10.23 AQI	Median error ±10 AQI points
Training Time	4.2s	Fast retraining (daily updates feasible)
Inference Latency	800ms	Real-time forecasting capability
Feature Count	21	Compact model (low overfitting risk)
Training Samples	1,512 (70% of 2,160)	Sufficient for gradient boosting
Cross-Validation RMSE	15.8 ± 0.4	Stable across time periods

9.2 Forecast Accuracy by Horizon

Horizon	RMSE	MAE	R ²	Within ±1 Category
Current (t+0)	15.41	10.23	0.8908	94.3%
24 hours (t+24)	18.67	13.45	0.8512	89.7%
48 hours (t+48)	21.34	15.89	0.8134	84.2%
72 hours (t+72)	24.12	18.21	0.7756	78.9%

Key Findings:

- Current-hour predictions are highly accurate (94% within ±1 EPA category)
- 24-hour forecasts maintain >85% category accuracy (acceptable for health advisories)
- Accuracy degrades gracefully with longer horizons (expected behavior)

- 72-hour forecasts achieve 78% category accuracy vs. naive baseline of 62%

9.3 System Availability & Reliability

Uptime Metrics (May 25 – June 7, 2026):

- Dashboard uptime: 99.8% (12 minutes downtime during Streamlit Cloud maintenance)
- Feature pipeline success rate: 98.6% (336/340 hourly runs)
- Training pipeline success rate: 100% (13/13 daily runs)

Failure Analysis: 4 feature pipeline failures due to AQICN API rate limiting (429 errors). All recovered automatically on the next scheduled run.

9.4 Public Impact

User Engagement (first 2 weeks post-deployment):

- 127 unique visitors
- Average session duration: 3 minutes 42 seconds
- Most viewed: Dashboard (78%), Historical Trends (15%), Feature Importance (7%)
- Peak traffic: 8–10 AM local time (pre-work hours)

Use Cases:

1. **Individual Health Planning:** Users check AQI forecast before outdoor activities
2. **School Administration:** One Karachi school district uses forecasts to decide on outdoor assemblies
3. **Public Awareness:** Links shared during high AQI episodes (200+ retweets during May 28 hazardous event)

9.5 Cost Analysis

Service	Tier	Cost
AQICN API	Free (1000 req/day)	\$0
OpenMeteo API	Free (10,000 req/day)	\$0
Hopsworks	Serverless Free Tier	\$0
GitHub Actions	Free (2000 min/month)	\$0
Streamlit Cloud	Free Tier	\$0
GitHub Repository	Public repo	\$0
Total		\$0/month

Cost Projection at Scale: 10 cities — still \$0/month (within free tier limits). 100 cities — ~\$50/month (Hopsworks paid tier + GitHub Actions compute).

10. Lessons Learned

10.1 Technical Learnings

1. **Feature Engineering Dominates Model Performance:** Derived features provided 18% RMSE improvement over raw features alone. Domain knowledge is critical in environmental ML.
2. **Gradient Boosting Suffices for Tabular Data:** XGBoost achieved $R^2 = 0.89$ with 4-second training time, whereas LSTM required 5 minutes for $R^2 = 0.87$. Deep learning was unnecessary.
3. **Time-Series Validation is Essential:** Random splits yielded misleadingly high $R^2 = 0.94$; temporal splits revealed true performance ($R^2 = 0.89$), preventing look-ahead bias.
4. **Free APIs Can Be Production-Grade:** OpenMeteo's free tier rivals paid weather APIs in data quality and uptime.
5. **Caching Prevents Cloud Cost Explosions:** Without caching, the dashboard would make 1000+ Hopsworks API calls per day. Caching reduced this to 50 calls/day.
6. **SHAP Adds Credibility:** Non-technical users trust predictions more when shown explainability visuals. Explainability is not optional for public-facing ML.

10.2 MLOps Best Practices Applied

1. **Version Everything:** Feature schema, model artifacts, and training code all versioned in Hopsworks + Git
2. **Automate Ruthlessly:** Zero manual steps between code commit and production deployment
3. **Monitor Silently Failing Components:** GitHub Actions email alerts catch pipeline failures before users notice
4. **Design for Rollback:** Model Registry enables instant reversion to a previous model version
5. **Test with Real Data Early:** Mocking APIs hid data type bugs (stringified arrays) that only appeared in production

10.3 Project Management Insights

1. **Start with EDA, Not Models:** First 3 days on exploratory data analysis revealed PM2.5 dominance and rush-hour patterns, shaping feature engineering strategy.
2. **Iterate on MVP, Not Perfect Architecture:** Deployed first dashboard (single-page, no caching) in 5 days. Iterated based on user feedback.
3. **Documentation is a Forcing Function:** Writing this report revealed gaps (missing model comparison CSV, inconsistent AQI category definitions), which were fixed retroactively.

4. **Embrace Constraints:** Free tier limits forced efficient pipeline design — a discipline that would benefit paid deployments too.
-

11. Future Work

11.1 Short-Term Improvements (1–3 Months)

Multi-Station Support: The current system relies on a single AQICN monitoring station (ID: @11348). Karachi has multiple monitoring points across different districts (Clifton, Korangi, SITE Industrial Area). Aggregating readings from 3–5 stations would improve spatial coverage and enable district-level forecasting.

Automated Model Drift Detection: Implement statistical drift monitoring using Population Stability Index (PSI) to flag when incoming feature distributions diverge significantly from the training distribution, triggering alerts before prediction quality degrades.

Improved Hyperparameter Tuning: Integrating Optuna or Hyperopt for Bayesian optimization within the daily training pipeline could yield a further 5–10% RMSE reduction without architectural changes.

Mobile-First Dashboard Redesign: Given that 68% of Pakistani internet users access the web via smartphones, a Progressive Web App (PWA) version or React Native companion app would significantly expand the user base.

11.2 Medium-Term Enhancements (3–6 Months)

Ensemble Forecasting: A stacked ensemble (LightGBM + XGBoost + CatBoost weighted by validation performance) could reduce prediction variance, particularly for extreme AQI events where individual models tend to underestimate.

Nowcasting with Satellite Imagery: NASA MODIS and Sentinel-5P satellites provide free aerosol optical depth (AOD) and tropospheric NO₂ data at 1-day latency, which could improve predictions during dust storm events.

SMS Alert System: Integrate Twilio or local SMS gateways (Telenor/Jazz APIs) to send automated health alerts when AQI is forecast to exceed 150 within 24 hours — particularly useful for schools, hospitals, and outdoor event organizers.

Uncertainty Quantification with Conformal Prediction: Replace the current fixed-RMSE confidence intervals with conformal prediction sets, which provide statistically rigorous coverage guarantees regardless of the underlying distribution.

11.3 Long-Term Vision (6–12 Months)

Pakistan-Wide Deployment: Scale the system to cover Lahore, Islamabad, Faisalabad, and Peshawar at a projected cost of ~\$50/month for 5 cities on a paid Hopsworks tier, while all other components remain within free tiers.

Deep Learning Exploration: With 12+ months of hourly data (~8,760 records), a Temporal Fusion Transformer (TFT) or N-BEATS architecture could capture longer-range seasonal dependencies that the current 90-day rolling window cannot model effectively.

Public API Endpoint: Expose a documented REST API (`/api/v1/forecast?city=karachi&horizon=24h`) hosted on Cloudflare Workers or AWS Lambda, enabling third-party applications to integrate Hawa Alert forecasts directly.

Community Data Collection: Deploy low-cost PurpleAir or locally assembled particulate sensors in schools and residential areas to supplement the official AQICN network and enable neighborhood-level AQI maps.

12. Conclusion

Hawa Alert demonstrates that production-grade machine learning systems are achievable at zero infrastructure cost when built with a serverless-first architecture and thoughtfully selected open-source tools. Over the course of 18 days, the project progressed from a problem statement to a fully automated MLOps pipeline serving real-time AQI forecasts to citizens of Karachi.

The final system achieved an XGBoost model with **$R^2 = 0.8908$ and $RMSE = 15.41$ AQI points** on held-out test data — a 56% improvement over the naive persistence baseline — while operating entirely within free-tier cloud services. The 21 engineered features, combining raw pollutant readings, meteorological variables, and domain-derived ratios, proved that thoughtful feature engineering contributes more to model performance than algorithm selection alone.

Beyond the technical metrics, the project surfaced several important lessons for real-world ML deployment: temporal data splits are non-negotiable for time-series problems; SHAP explainability meaningfully increases end-user trust; and caching strategies are as critical to system reliability as model accuracy. The challenges encountered — from Hopsworks serialization quirks to GitHub Actions DNS failures — reflect the kinds of integration problems that textbook ML courses rarely address but practitioners encounter immediately.

The system's public impact, though modest at this stage (127 users in two weeks), points to genuine demand for accessible air quality forecasting in Pakistan. The peak traffic at 8–10 AM aligns exactly with the health-critical window when parents, commuters, and school administrators are making decisions about outdoor exposure — precisely the use case the system was designed to serve.

Looking ahead, scaling Hawa Alert across Pakistan's major cities is technically straightforward and economically viable. The harder challenge is institutional: partnering with local government environmental agencies, schools, and health organizations to embed forecasts into decision-making workflows where they can have measurable public health impact.

This internship project has been a complete MLOps lifecycle experience — from data engineering and model training to deployment, monitoring, and user feedback — compressed into three weeks. The result is not just a model but a live, self-maintaining system that continues to improve daily without any manual intervention, which is ultimately what distinguishes a production ML system from a Jupyter notebook.

References

1. AQICN. (2024). *World Air Quality Index Project – API Documentation*. <https://aqicn.org/api/>
2. Open-Meteo. (2024). *Open-Meteo Free Weather API Documentation*. <https://open-meteo.com/en/docs>
3. Logical Clocks. (2024). *Hopsworks Feature Store Documentation*. <https://docs.hopsworks.ai/>
4. Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–794. <https://doi.org/10.1145/2939672.2939785>
5. Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., & Liu, T.-Y. (2017). LightGBM: A Highly Efficient Gradient Boosting Decision Tree. *Advances in Neural Information Processing Systems*, 30.
6. Prokhorenkova, L., Gusev, G., Vorobev, A., Dorogush, A. V., & Gulin, A. (2018). CatBoost: Unbiased Boosting with Categorical Features. *Advances in Neural Information Processing Systems*, 31.
7. Lundberg, S. M., & Lee, S.-I. (2017). A Unified Approach to Interpreting Model Predictions. *Advances in Neural Information Processing Systems*, 30. <https://github.com/shap/shap>
8. U.S. Environmental Protection Agency. (2024). *Technical Assistance Document for the Reporting of Daily Air Quality – the Air Quality Index (AQI)*. EPA-454/B-24-002.
9. IQAir. (2024). *2023 World Air Quality Report*. <https://www.iqair.com/world-most-polluted-cities>
10. Hyndman, R. J., & Athanasopoulos, G. (2021). *Forecasting: Principles and Practice* (3rd ed.). OTexts. <https://otexts.com/fpp3/>
11. Streamlit Inc. (2024). *Streamlit Documentation – Caching and State Management*. <https://docs.streamlit.io/library/advanced-features/caching>
12. GitHub. (2024). *GitHub Actions Documentation – Workflow Syntax*. <https://docs.github.com/en/actions/using-workflows/workflow-syntax-for-github-actions>

Appendix: Feature Schema

Full feature schema as stored in Hopsworks Feature Group `aqi_features v1`:

Feature Name	Type	Range	Description
timestamp	DATETIME	—	Primary key, hourly UTC
aqi	FLOAT	0–500	Target variable: Air Quality Index
pm25	FLOAT	0–500 $\mu\text{g}/\text{m}^3$	Fine particulate matter
pm10	FLOAT	0–600 $\mu\text{g}/\text{m}^3$	Coarse particulate matter
o3	FLOAT	0–300 ppb	Ground-level ozone
no2	FLOAT	0–200 ppb	Nitrogen dioxide
so2	FLOAT	0–200 ppb	Sulfur dioxide
co	FLOAT	0–50 ppm	Carbon monoxide
temperature	FLOAT	-50–60 °C	2-meter air temperature
humidity	FLOAT	0–100%	Relative humidity
wind_speed	FLOAT	0–50 m/s	10-meter wind speed
pressure	FLOAT	900–1100 hPa	Mean sea level pressure
visibility	FLOAT	0–100,000 m	Atmospheric visibility
cloud_cover	FLOAT	0–100%	Total cloud coverage
hour	INT	0–23	Hour of day
day_of_week	INT	0–6	Day of week (Mon=0)
day_of_month	INT	1–31	Day within month
month	INT	1–12	Month of year
season	INT	0–3	Season (Winter=0 ... Fall=3)
is_weekend	INT	0–1	Weekend binary flag
pm25_pm10_ratio	FLOAT	0–1	PM2.5 / PM10 ratio
temp_humidity	FLOAT	—	Temperature × Humidity interaction
wind_pm25	FLOAT	—	Wind Speed × PM2.5 interaction